

# 详细介绍一下用CRT实现大模数NTT的原理

用 CRT 实现大模数 NTT，本质是：不直接在“大模数”下做 NTT，而是在多个小的 NTT 友好质数下分别做 NTT，最后用中国剩余定理把结果拼回大模数结果。

下面按原理讲清楚。

## 1. 为什么需要 CRT

普通 NTT 需要模数  $p$  满足：

$$p = k * N + 1$$

其中  $N$  是变换长度，保证模  $p$  下存在  $N$  次单位根  $\omega$ 。

但实际问题里目标模数  $M$  可能很大，比如：

$$M = 10^{18} + 7$$

$$M = 2^{64}$$

$$M = \text{某个密码学大模数 } q$$

这些模数往往有几个问题：

- 不一定存在合适的  $N$  次原根；
- 模乘可能溢出；
- 模数不是质数，NTT 理论不方便；
- 即使能做，效率也低。

所以做法是选若干个小质数：

$$p_1, p_2, \dots, p_k$$

每个都适合 NTT，例如：

$$p_i = c_i * N + 1$$

然后分别在这些模数下做 NTT。

## 2. CRT 的数学基础

中国剩余定理说，如果  $p_1, p_2, \dots, p_k$  两两互质，那么模

$$P = p_1 * p_2 * \dots * p_k$$

的一个数  $x$  可以唯一对应到一组余数：

$$x \bmod p_1$$

$$x \bmod p_2$$

$$\dots$$

$$x \bmod p_k$$

也就是：

$$\mathbb{Z} / P\mathbb{Z} \cong \mathbb{Z} / p_1\mathbb{Z} \times \mathbb{Z} / p_2\mathbb{Z} \times \dots \times \mathbb{Z} / p_k\mathbb{Z}$$

这意味着，如果我们想算：

$$C = A * B \bmod P$$

可以分别算：

$$C_1 = A * B \bmod p_1$$

$$C_2 = A * B \bmod p_2$$

$$\dots$$

$$C_k = A * B \bmod p_k$$

然后用 CRT 还原出：

$$C \bmod P$$

多项式卷积也是一样的，因为加法、乘法都可以按模数分量独立进行。

## 3. 用多个小模数做大模数卷积

假设要计算两个多项式：

$$A(x) = a_0 + a_1 x + \dots + a_{n-1} x^{(n-1)}$$

$$B(x) = b_0 + b_1 x + \dots + b_{m-1} x^{(m-1)}$$

目标是得到：

$$C(x) = A(x)B(x) \bmod M$$

步骤如下。

### 第一步：选择 NTT 长度

令：

$$L \geq n + m - 1$$

通常取 2 的幂：

$$L = 2^t$$

### 第二步：选择多个 NTT 质数

选择若干个质数  $p_1, p_2, \dots, p_k$ ，满足：

$$p_i = c_i * L + 1$$

这样每个  $p_i$  下都有  $L$  次单位根，可以做长度为  $L$  的 NTT。

常见 NTT 质数有：

$$998244353 = 119 * 2^{23} + 1$$

$$1004535809 = 479 * 2^{21} + 1$$

$$469762049 = 7 * 2^{26} + 1$$

### 第三步：分别取模

对每个  $p_i$ ，把系数映射过去：

$$a_j^{(i)} = a_j \bmod p_i$$

$$b_j^{(i)} = b_j \bmod p_i$$

然后在模  $p_i$  下做 NTT:

$$A_i = \text{NTT}_{p_i}(A)$$

$$B_i = \text{NTT}_{p_i}(B)$$

点值相乘:

$$C_i[t] = A_i[t] * B_i[t] \bmod p_i$$

再做逆 NTT:

$$c_j^{(i)} = \text{INTT}_{p_i}(C_i)$$

这样得到每个卷积系数在模  $p_i$  下的值:

$$c_j \bmod p_i$$

#### 第四步: CRT 还原

对于每个卷积系数  $c_j$ , 已知:

$$c_j \equiv c_j^{(1)} \bmod p_1$$

$$c_j \equiv c_j^{(2)} \bmod p_2$$

...

$$c_j \equiv c_j^{(k)} \bmod p_k$$

用 CRT 还原出:

$$c_j \bmod P$$

其中:

$$P = p_1 * p_2 * \dots * p_k$$

如果最终目标是模  $M$ ，再做：

$$c_j \bmod M$$

#### 4. 关键条件：小模数乘积要足够大

如果目标只是算模  $P$ ，那当然直接 CRT 回  $P$ 。

但如果目标是算模某个大模数  $M$ ，通常有两种情况。

##### 情况一： $M$ 本身可以分解成 NTT 友好因子

如果：

$$M = p_1 * p_2 * \dots * p_k$$

且每个  $p_i$  都适合 NTT，那么最自然。直接在每个  $p_i$  下做 NTT，再 CRT 回  $M$ 。

这种情况最漂亮，因为你真的在计算：

$$A(x)B(x) \bmod M$$

##### 情况二： $M$ 不是这些小质数的乘积

更常见的是， $M$  不适合 NTT。那就选一组辅助质数  $p_i$ ，令：

$$P = p_1 * p_2 * \dots * p_k$$

只要  $P$  足够大，可以唯一覆盖真实整数卷积的范围，就可以先还原出真实整数结果，再对  $M$  取模。

例如如果：

$$0 \leq a_i, b_i < M$$

卷积中每个系数最多大约是：

$$L * (M - 1)^2$$

所以要保证：

$$P > L * (M - 1)^2$$

这样 CRT 还原出来的结果不会发生混淆。

如果使用有符号系数，通常需要保证：

$$P > 2 * \text{最大绝对值界}$$

因为 CRT 默认还原到  $[0, P)$ ，还需要判断它代表正数还是负数。

## 5. 为什么 NTT 可以逐模数做

NTT 本质上是在某个环里做点值变换。模  $P$  下的多项式运算，可以通过 CRT 分解成多个模  $p_i$  下的运算。

如果：

$$P = p_1 * p_2 * \dots * p_k$$

那么：

$$R_P = (Z / PZ)[x]$$

可以分解成：

$$R_{p_1} \times R_{p_2} \times \dots \times R_{p_k}$$

多项式加法、乘法、点值乘法都逐分量进行。

也就是说：

$$\text{NTT}_P(A)$$

虽然可能不好直接做，但它对应于：

```
NTT_p1(A mod p1)
NTT_p2(A mod p2)
...
NTT_pk(A mod pk)
```

最后用 CRT 把每个系数拼回来即可。

## 6. CRT 还原公式

设：

```
P = p1 * p2 * ... * pk
Pi = P / pi
```

求：

```
inv_i = Pi^(-1) mod pi
```

那么：

```
x ≡ Σ ri * Pi * inv_i mod P
```

其中  $ri = x \bmod pi$ 。

也就是：

```
x = (r1 * P1 * inv1
    + r2 * P2 * inv2
    + ...
    + rk * Pk * invk) mod P
```

实际实现时，如果  $P$  很大，直接乘可能溢出，需要用大整数、128 位整数、Montgomery、Barrett，或者逐步 CRT。

## 7. 三模数 CRT 的常见写法

在竞赛或工程里常用三个 30 位左右的 NTT 质数。比如：

$$p1 = 998244353$$

$$p2 = 1004535809$$

$$p3 = 469762049$$

它们的乘积约为：

$$4.7 * 10^{26}$$

足以覆盖很多整数卷积场景。

对每个系数拿到：

$$r1, r2, r3$$

然后还原出  $x \bmod M$ 。

可以先合并前两个：

$$x12 = r1 + p1 * t$$

其中：

$$t \equiv (r2 - r1) * p1^{(-1)} \bmod p2$$

于是：

$$x12 \equiv r1 \bmod p1$$

$$x12 \equiv r2 \bmod p2$$

再合并第三个：

$$x = x12 + p1 * p2 * u$$

其中：

$$u \equiv (r3 - x12) * (p1 * p2)^{(-1)} \bmod p3$$



最后：

```
answer = x mod M
```

这种逐步合并比直接套大公式更适合避免溢出。

## 8. 负数系数的处理

CRT 还原默认得到的是：

$$0 \leq x < P$$

但真实卷积系数可能是负数。如果已知真实值范围满足：

$$-B \leq x \leq B$$

且：

$$P > 2B$$

那么可以这样转回有符号值：

```
if x > P / 2:  
    x -= P
```

然后再对目标模数  $M$  取模。

## 9. 与直接大模数 NTT 的区别

直接大模数 NTT 要求目标模数  $M$  下存在合适单位根，且运算可控。

CRT-NTT 不要求  $M$  适合 NTT。它只要求辅助质数适合 NTT，并且它们的乘积足够大。

所以 CRT-NTT 的核心优势是：

把一个困难的大模数卷积  
变成多个简单的小模数卷积  
再用 CRT 拼回来

代价是：

- 要做多次 NTT；
- 要做 CRT 还原；
- 实现更复杂；
- 需要小心溢出和系数范围。

## 10. 一句话总结

用 CRT 实现大模数 NTT，就是选择多个形如  $k * N + 1$  的小质数，在每个小模数下独立完成 NTT 卷积，然后利用中国剩余定理把每个系数的多个余数合并，得到大模数下或整数意义下的卷积结果。它绕开了大模数不适合 NTT 的问题，是实现任意大模数多项式乘法的常用方法。

**我在用CRT实现NTT的基础上实现并行优化，我目前有两个思路，一个是对小模数进行并行，一个是小模数串行但是蝴蝶变换的时候并行，在不考虑并行嵌套的情况下这两种的区别在哪，哪个更好，如果考虑并行嵌套又该怎么做**

结论先说：只做一层并行时，优先级通常是：小模数并行更简单、同步少；蝴蝶并行扩展性更强，但同步和内存带宽压力更大。哪个更好取决于 小模数个数  $k$ 、NTT 长度  $N$ 、线程数  $P$ 。

### 1. 小模数并行

CRT-NTT 里每个小模数完全独立：

```
mod p1: NTT(A), NTT(B), 点乘, INTT
mod p2: NTT(A), NTT(B), 点乘, INTT
...
```

所以可以让不同线程处理不同模数。

优点：

- 粒度粗，任务大，调度开销低；
- 每个模数内部串行，不需要每一层蝴蝶都同步；
- 实现简单，线程之间几乎没有共享写；
- 很适合  $k$  接近核心数、 $N$  不太大时。

缺点：

- 并行度上限是小模数个数  $k$ ；
- CRT 常用小模数一般只有 2 到 4 个，所以 8 核、16 核机器会浪费很多核心；
- 多个模数同时跑会同时扫大数组，容易打满内存带宽。

时间模型大概是：

$$T_{\text{mod\_parallel}} \approx \text{ceil}(k / P) * T_{\text{single\_NTT\_mod}}$$

最大加速比基本不超过  $k$ 。

所以如果你有 3 个模数、16 个核心，单靠小模数并行最多也就吃满 3 个核心左右。

## 2. 蝴蝶变换并行

另一种是每个小模数串行处理，但在每一层 NTT 里并行执行蝴蝶。

一层 NTT 中，所有蝴蝶都是独立的：

```
for stage in logN:
    parallel_for each butterfly in this stage
    barrier
```

优点：

- 并行度可以到  $O(N)$ ，适合大长度 NTT；
- 能吃满很多核心；

- GPU 或大规模 SIMD 场景下通常必须走这条路。

缺点：

- 每一层都需要同步，NTT 有  $\log N$  层；
- $N$  小时，同步开销可能比计算还明显；
- 内存访问和 cache 行为更敏感；
- 实现比小模数并行复杂。

时间模型大概是：

$$T_{\text{butterfly\_parallel}} \approx k * (T_{\text{single\_NTT\_mod}} / P + \text{sync\_cost} * \log N)$$

所以它适合：

$N$  很大， $P$  明显大于  $k$

例如  $N = 2^{20}$ 、 $k = 3$ 、 $P = 16$ ，只做小模数并行肯定不够，蝴蝶并行会更有意义。

### 3. 不考虑嵌套时，哪个更好

如果只能选一种，我会这样判断：

核心数  $P \leq$  小模数个数  $k$ ：

小模数并行通常更好，简单且同步少

核心数  $P \gg$  小模数个数  $k$ ，且  $N$  很大：

蝴蝶并行通常更好，可以吃满核心

$N$  较小：

小模数并行通常更稳，蝴蝶并行同步成本偏高

$N$  很大：

蝴蝶并行更有扩展性，但要注意内存带宽

在普通 CRT-NTT 里， $k$  通常只有 2、3、4，所以在现代多核 CPU 上，**单独的小模数并行往往不够用**。但如果只是 2 到 4 核，或者  $N$  不大，小模数并行会很舒服。

### 4. 考虑并行嵌套时怎么做

嵌套的目标是同时利用两层并行：

模数之间并行  
模数内部的蝴蝶并行

但关键原则是：**不要真的让每个模数都开一套完整线程池，否则容易线程过量竞争。**

更好的做法是使用一个全局线程池，把任务拆成：

```
(modulus_id, stage_id, butterfly_chunk)
```

也就是在每一层 stage 上，对所有模数的蝴蝶块一起调度。

例如：

```
for stage in stages:
    parallel_for over (modulus, butterfly_chunk):
        do butterflies for that modulus and chunk
    barrier
```

这样有几个好处：

- 不会嵌套创建线程；
- 负载均衡更好；
- 同一套线程池控制总并行度；
- 可以自然覆盖  $k * N/2$  个蝴蝶任务。

对于完整卷积，可以按阶段做：

1. 对 A 的所有模数做 forward NTT
2. 对 B 的所有模数做 forward NTT
3. 对所有模数做点乘
4. 对所有模数做 inverse NTT
5. 对每个系数并行 CRT 重构

其中第 1、2、4 步可以用：

```
parallel over modulus + butterfly chunks
```

第 3 步可以用：

parallel over modulus + index

第 5 步可以用：

parallel over coefficient index

## 5. 实际推荐策略

比较稳的工程策略是自适应：

if N 小：

只做小模数并行

else if  $P \leq k$ ：

只做小模数并行

else：

做模数 + 蝴蝶混合并行

比如：

$k = 3, P = 16$

可以让 3 个模数同时跑，每个模数内部再分到若干线程；但实现上最好仍然用一个全局线程池，而不是 3 个嵌套并行区。

可以粗略理解为：

总并行度  $\approx k * \text{每个模数内部并行度}$

但受限于：

总线程数  $\leq P$

内存带宽

同步开销

cache 容量

## 6. 我的建议

如果你现在是在 CPU 上实现：

- 第一版：先做小模数并行，最容易正确，收益稳定；
- 第二版：给单个 NTT 加 stage 内 `parallel_for` ；
- 最终版：统一线程池，按（模数，蝴蝶块）混合调度；
- CRT 重构也别忘了并行，它是天然按系数独立的。

一句话判断：**小模数并行是低成本粗粒度并行，适合起步；蝴蝶并行是高扩展细粒度并行，适合大 N 和多核；最终高性能实现通常要两者结合，但必须用统一调度避免并行嵌套失控。**

## 在CRT蝴蝶变换的并行过程中如何保证负载均衡

在 CRT 版本里，“蝶形变换并行”主要发生在每个小模数 NTT 内部。要保证负载均衡，关键是不要只按外层 block 粗略分配，而要根据当前 NTT 层的任务数量选择合适粒度。

NTT 每一层的蝶形总数基本固定：

```
total = limit / 2
```

但 block 数会变化：

```
blocks = limit / len
```

前几层 `len` 小，`blocks` 很多，直接按 block 分配给线程就比较均衡；后几层 `len` 大，`blocks` 可能少于线程数，如果仍然按 block 分配，就会出现很多线程没活干。

所以比较合理的做法是分两种情况。

### 1. block 数足够时，按 block 静态划分

```
if (blocks >= thread_count) {  
    // 每个线程负责若干完整 block  
}
```

此时每个 block 的工作量相同，都是 `len / 2` 个蝶形，所以按 block 平均分配即可。比如：

```
int l = blocks * tid / thread_count;
int r = blocks * (tid + 1) / thread_count;
```

这样每个线程拿到的 block 数最多只差 1，负载比较均衡。

## 2. block 数不足时，按 butterfly 总数划分

当：

```
blocks < thread_count
```

如果继续按 block 分，就会浪费线程。此时应该把这一层所有蝶形看成一个一维任务数组：

```
total = limit / 2
```

然后按线程编号均分：

```
int l = total * tid / thread_count;
int r = total * (tid + 1) / thread_count;
```

每个线程处理  $[l, r)$  中的 butterfly。对于某个全局 butterfly 编号  $idx$ ，再映射回：

```
int block = idx / half;
int k = idx % half;
int base = block * len;
```

对应操作：

```
u64 x = a[base + k];
u64 y = mul_mod(a[base + k + half], w, mod);
a[base + k] = add_mod(x, y, mod);
a[base + k + half] = sub_mod(x, y, mod);
```

这样即使后几层只有 1 个 block，也能把这个 block 内部的蝶形拆给多个线程，避免只有一个线程工作。

## 为什么这样能保证负载均衡

因为每个蝶形运算的计算量基本相同，都包含：



一次模乘  
一次模加  
一次模减  
若干数组访问

所以按 butterfly 数量均分时，每个线程的任务量基本一致。用：

```
l = total * tid / thread_count  
r = total * (tid + 1) / thread_count
```

还能处理 total 不能整除线程数的情况，每个线程任务数最多相差 1。

## CRT 层面的负载均衡

CRT 有多个小模数，每个小模数下都要做一次完整 NTT。模数并行时，可以让不同线程处理不同模数：

```
#pragma omp parallel for schedule(static)  
for (int i = 0; i < CRT_CNT; ++i) {  
    multiply_mod_ntt(a, b, residues[i], n, CRT_MODS[i]);  
}
```

因为四个小模数的 NTT 长度相同，算法流程相同，所以每个模数任务计算量接近，静态分配即可。

但四模数最多只有 4 个任务，如果平台有 8 核，模数并行最多只能用 4 个线程。为了更充分利用核心，可以采用：

模数并行 + 每个模数内部蝶形并行

不过要注意控制总线程数，避免嵌套并行导致线程数超过 8。

## 同步如何处理

负载均衡之外，还要保证每一层 NTT 完成后才能进入下一层。因为第  $s+1$  层依赖第  $s$  层的结果。

Pthread 版本需要在每层末尾使用：

```
pthread_barrier_wait(&barrier);
```

OpenMP 的：

```
#pragma omp parallel for
```

在循环结束时默认有隐式同步，也能保证这一层所有蝶形完成后再进入下一层。

所以总结一下：

前期 block 多：按 block 均分  
后期 block 少：按 butterfly 总数均分  
每层结束：barrier 或 OpenMP 隐式同步  
CRT 合并：按输出系数均分

这样既能保证正确性，又能尽量避免线程空闲，实现较好的负载均衡。

## 介绍一下Openmp的SIMD自动优化

OpenMP 的 SIMD 优化，核心是告诉编译器：**这个循环的不同迭代之间可以安全地用 SIMD 向量指令并行执行**。它不是线程级并行，而是**单核内部的数据级并行**。

也就是说：

OpenMP parallel for -> 多线程并行  
OpenMP simd -> 单线程内向量化  
OpenMP parallel for simd -> 多线程 + 每个线程内部 SIMD

### 1. SIMD 是什么

SIMD 是 Single Instruction Multiple Data，即“一条指令处理多个数据”。

例如普通标量循环：

```
for (int i = 0; i < n; i++) {  
    c[i] = a[i] + b[i];  
}
```

标量执行一次处理一个元素：

```
c[0] = a[0] + b[0]
c[1] = a[1] + b[1]
c[2] = a[2] + b[2]
...
```

如果 CPU 支持 AVX2，一条 256-bit 指令可以一次处理 8 个 float 或 4 个 double：

```
c[i:i+7] = a[i:i+7] + b[i:i+7]
```

这就是 SIMD 向量化。

## 2. 编译器本来就会自动向量化

现代编译器在 -O2 / -O3 下会尝试自动向量化，比如 GCC、Clang、ICC、MSVC 都会分析循环。

但编译器很保守。如果它不确定以下问题，就可能不向量化：

- 指针是否别名；
- 循环迭代之间是否有依赖；
- 数组是否对齐；
- 循环边界是否规整；
- 是否有分支；
- 是否有函数调用；
- 浮点重排是否会改变结果。

例如：

```
void f(double* a, double* b, double* c, int n) {
    for (int i = 0; i < n; i++) {
        a[i] = b[i] + c[i];
    }
}
```

编译器可能担心：

a、b、c 指向重叠内存

如果重叠，就可能存在读写依赖，向量化不安全。

### 3. OpenMP SIMD 的作用

OpenMP SIMD 指令用于给编译器额外保证：

```
#pragma omp simd
for (int i = 0; i < n; i++) {
    a[i] = b[i] + c[i];
}
```

意思是：

程序员保证这个循环的各次迭代可以 SIMD 并行执行

编译器就可以更大胆地生成向量指令。

注意：#pragma omp simd 通常不会启动多个线程。它只是影响当前线程内的向量化。

### 4. 常见写法

基础写法：

```
#pragma omp simd
for (int i = 0; i < n; i++) {
    c[i] = a[i] + b[i];
}
```

如果要做归约：

```
double sum = 0.0;

#pragma omp simd reduction(+:sum)
for (int i = 0; i < n; i++) {
    sum += a[i] * b[i];
}
```

如果没有 reduction，这种循环可能因为 sum 有循环依赖而无法安全向量化。

## 5. simd 和 parallel for 的区别

这两个经常被混淆。

```
#pragma omp parallel for
for (int i = 0; i < n; i++) {
    c[i] = a[i] + b[i];
}
```

这是多线程，每个线程处理一部分  $i$ 。

```
#pragma omp simd
for (int i = 0; i < n; i++) {
    c[i] = a[i] + b[i];
}
```

这是 SIMD，通常还是一个线程，但一条指令处理多个元素。

两者可以组合：

```
#pragma omp parallel for simd
for (int i = 0; i < n; i++) {
    c[i] = a[i] + b[i];
}
```

含义是：

先把循环分给多个线程；  
每个线程内部再对自己的循环片段做 SIMD。

这对 NTT 的大数组点乘、CRT 重构、模加模减等阶段很有用。

## 6. OpenMP SIMD 的重要子句

**reduction**

用于归约：

```
#pragma omp simd reduction(+:sum)
for (int i = 0; i < n; i++) {
    sum += x[i];
}
```

支持常见操作：

+, \*, -, &, |, ^, &&, ||

### **aligned**

告诉编译器指针按某个字节数对齐：

```
#pragma omp simd aligned(a, b, c: 32)
for (int i = 0; i < n; i++) {
    c[i] = a[i] + b[i];
}
```

这表示 a、b、c 至少 32 字节对齐，适合 AVX。

如果你声明错了，可能导致未定义行为或性能异常，所以只在确定对齐时使用。

### **safelen**

指定不会产生循环依赖的最小距离：

```
#pragma omp simd safelen(8)
for (int i = 0; i < n; i++) {
    a[i] = a[i + 8] + 1;
}
```

它告诉编译器向量长度不要超过某个安全范围。

### **simdlen**

建议使用的 SIMD lane 数：

```
#pragma omp simd simdlen(4)
for (int i = 0; i < n; i++) {
    c[i] = a[i] * b[i];
}
```

例如 double 下 simdlen(4) 对应 256-bit AVX。

这只是建议，最终由编译器和目标架构决定。

## linear

用于循环里有线性变化的变量：

```
int idx = base;

#pragma omp simd linear(idx:1)
for (int i = 0; i < n; i++) {
    a[idx] = b[i];
    idx++;
}
```

表示 idx 每次迭代递增 1。

## 7. 在 NTT 里哪些地方适合 SIMD

CRT-NTT 中，SIMD 最容易用在这些地方：

```
for (int i = 0; i < n; i++) {
    a[i] = a[i] * b[i] % mod;
}
```

点乘阶段非常规整，适合：

```
#pragma omp simd
for (int i = 0; i < n; i++) {
    a[i] = mul_mod(a[i], b[i], mod);
}
```

CRT 重构也适合按系数 SIMD 或至少多线程：

```
#pragma omp parallel for simd
for (int i = 0; i < n; i++) {
    ans[i] = crt(r1[i], r2[i], r3[i]);
}
```

蝴蝶阶段能不能 SIMD，要看数据布局和实现方式。典型蝴蝶：

```
u = a[i];
v = a[i + half] * w % mod;
a[i] = u + v;
a[i + half] = u - v;
```

如果  $j$  连续， $a[i]$  和  $a[i + half]$  都是连续访问，这一段可以尝试 SIMD：

```
for (int block = 0; block < n; block += len) {
    #pragma omp simd
    for (int j = 0; j < half; j++) {
        int i = block + j;
        butterfly(a[i], a[i + half], roots[j]);
    }
}
```

不过模乘  $\% \text{mod}$  可能比较难自动向量化。若想高性能，通常会用 Barrett 或 Montgomery，把  $\%$  换成更容易向量化的乘法、移位、减法。

## 8. OpenMP SIMD 不一定真的成功向量化

`#pragma omp simd` 是强提示，但不等于一定生成理想 SIMD。

可能失败的原因：

- 循环里有复杂函数调用；
- 模运算  $\%$  编译器不好向量化；
- 存在无法证明的内存依赖；
- 数据访问不连续；
- 分支太复杂；
- 目标编译参数没有开启对应指令集。



建议编译时打开向量化报告。

GCC:

```
g++ -O3 -fopenmp -march=native -fopt-info-vec-optimized -fopt-info-vec-missed
```

Clang:

```
clang++ -O3 -fopenmp -march=native -Rpass=loop-vectorize -Rpass-missed=loop-vectorize
```

ICC/ICX:

```
icx -O3 -qopenmp -march=native -qopt-report=5
```

## 9. 对 CRT-NTT 的实践建议

如果你要在 CRT-NTT 中用 OpenMP SIMD，我建议顺序是：

1. 先给点乘阶段加 `parallel for simd`；
2. 再给数组拷贝、取模、归一化、CRT 重构加 SIMD；
3. 再尝试蝴蝶内层 `j` 循环 SIMD；
4. 把模乘实现改成 Barrett/Montgomery，减少 %；
5. 用编译器向量化报告确认是否真的生效；
6. 最后 benchmark，不要只看理论。

简单说：

```
#pragma omp simd
```

是你告诉编译器“这个循环可以向量化”的契约；

而真正的性能，还取决于循环结构、内存连续性、模乘实现和编译器能否生成高质量向量指令。